

Simulador Curiosity NASA

Juan Andrés Bravo

Daniel Leonardo Buitrago

Maria Jose Zabala

Pontificia Universidad Javeriana

Departamento de Ingenieria de Sistemas

Estructuras de Datos

Jorge Valenzuela

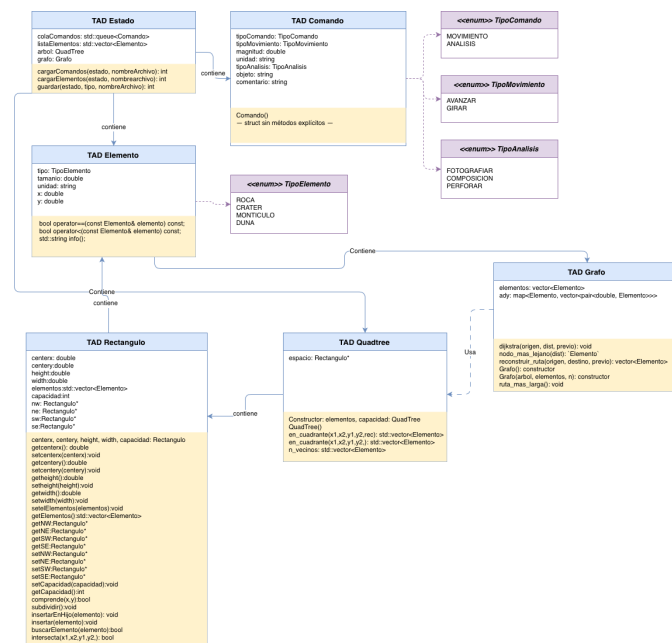
Bogotá, 2026

1. Tipos Abstractos de Datos (TADs)

Los tipos abstractos de datos definidos a continuación son usados a lo largo del sistema. El TAD Estado actúa como el contenedor central del sistema, agrupando la cola de comandos, la lista de elementos que el Robot Curiosity maneja durante su operación y el árbol espacial. El TAD Comando representa cada instrucción que se envía al robot, ya sea de movimiento o de análisis del terreno. El TAD Elemento representa cada componente físico identificado en el suelo con su ubicación geográfica. Para la administración espacial, se utiliza el TAD Rectangulo, que define la unidad base geométrica limitando un área del terreno con capacidad de subdividirse en subcuadrantes. A su vez, el TAD QuadTree es la estructura jerárquica que utiliza estos rectángulos para indexar los elementos geográficamente, permitiendo consultas espaciales eficientes. Todos los valores numéricos almacenados en estos TADs se encuentran normalizados y estandarizados: las distancias en metros y los ángulos en grados. Por otro lado, el TAD Grafo representa el mapa geográfico del terreno como una estructura de conexiones ponderadas entre elementos, donde cada arista refleja la distancia euclidiana entre dos puntos de interés. La construcción del TAD Grafo se apoya en el TAD Quadtree para lograr identificar los vecinos más cercanos de cada elemento y determinar la ruta más larga dentro del mapa, identificando los dos puntos de interés más alejados entre si según las conexiones generadas.

Figura 1.

Diagrama de relación — TADs del sistema Curiosity



Fuente: Elaboración propia mediante *app.diagrams.net*

2. Estructuras De Datos Utilizadas

a. Estructuras Lineales

Para el almacenamiento y manejo de información en memoria, el sistema hace uso de dos estructuras lineales fundamentales que forman parte del TAD Estado: una cola y un vector.

Para los comandos de desplazamiento se utilizó una cola. Esta elección responde directamente al comportamiento esperado del robot Curiosity: los comandos deben ejecutarse en el mismo orden en que fueron enviados, es decir, el primero en ser cargado es el primero en ejecutarse. Este comportamiento FIFO es precisamente la característica principal de una cola, lo que la convierte en la estructura más adecuada para este propósito. Adicionalmente, la cola garantiza que durante la simulación no se altere el orden original de los comandos. Ya que se trabaja sobre una copia de la estructura original.

Para los elementos de interés del terreno se utilizó un vector. A diferencia de los comandos, los elementos no tienen un orden de ejecución, sino que necesitan ser recorridos de forma secuencial para operaciones como guardar en archivo o ser procesados por estructuras jerárquicas en entregas futuras. El vector permite agregar elementos de forma dinámica sin conocer de antemano cuantos hay en el archivo, y facilita el recorrido completo de la colección. Su tamaño se ajusta automáticamente a medida que se cargan los datos.

Ambas estructuras se encuentran encapsuladas dentro del TAD Estado, evitando uso de variables globales y permitiendo que toda la información del sistema viaje de forma organizada entre los diferentes módulos mediante una referencia a dicho estado.

b. Estructuras Jerárquicas usadas

Para el almacenamiento, indexación geométrica y análisis espacial de los puntos de interés en la superficie marciana, el sistema hace uso de una estructura jerárquica principal que forma parte del TAD Estado: un Árbol Cuaternario (QuadTree).

Para la ubicación geográfica de los elementos del terreno se utilizó un QuadTree basado en nodos de tipo Rectángulo. Esta elección responde directamente a la necesidad del sistema de facilitar la planificación de desplazamientos del robot Curiosity mediante consultas espaciales. A diferencia del vector (que requiere un recorrido secuencial), el QuadTree organiza el espacio bidimensional de forma eficiente. Cuando se ubican los elementos, la estructura define un área total inicial y, si un cuadrante supera su capacidad

máxima, se subdivide recursivamente en cuatro subcuadrantes (Noroeste, Noreste, Suroeste, Sureste).

Esta estructura permite en la estructura ideal para las consultas de elementos dado un cuadrante específico, debido a su comportamiento de ramificación sectorizada generando las consultas de “en_cuadrante” de la manera más eficiente posible evitando recorrer la totalidad de los elementos.

c. Estructuras no Lineales usadas

Para el comando de ruta *_mas_larga* se utilizó una representación con grafos para interpretar los mapas con coeficiente de conectividad, utilizando la estructura del Quadtree para construir relaciones únicamente con los elementos más cercanos los unos a otros.

El grafo fue construido inicialmente obteniendo los n elementos más cercanos a cada elemento, considerando así el coeficiente de conectividad como un número que determina la cantidad de relaciones de cada elemento dentro del grafo.

Para obtener la ruta más larga se utilizó el algoritmo de Dijkstra invertido permitiendo obtener la ruta más larga de un nodo a otro, iterando consecutivamente entre cada uno de los nodos se obtiene la distancia más larga posible para el grafo construido.

3. Descripción de operaciones

A. CargarComandos

Campo	Descripción
Firma	int cargarComandos (Estado& estado, const std::string& nombreArchivo)
Entradas	estado: referencia al estado global del sistema nombreArchivo: ruta del archivo de texto con los comandos
Salida	int: -1 si el archivo no existe, 0 si esta vacío, n > 0 si cargo n comandos.
Precondiciones	El archivo debe existir y tener el formato correcto (tipo magnitud, unidad, tipo objeto 'comentario')
Postcondiciones	Estado.colasComandos queda reemplazada con los comandos del archivo, normalizados a metros y grados

B. CargarElementos

Campo	Descripción
Firma	int cargarElementos (Estado& estado, const std::string& nombreArchivo)
Entradas	estado: referencia al estado global del sistema nombreArchivo: ruta del archivo de texto con los comandos
Salida	int: -1 si error, 0 si esta vacío, n > 0 si cargo n elementos
Precondiciones	El archivo debe existir. Cada línea debe tener: tipo tamaño, unidad, coordX, coordY
Postcondiciones	Estado.listaElementos queda reemplazada. Tamanos normalizados a metros

C. Guardar

Campo	Descripción
Firma	int guardar (Estado& estado, const std::string& tipo, const std::string& nombreArchivo)
Entradas	estado: referencia al estado global tipo: 'comandos' o 'elementos' nombreArchivo: ruta del archivo de texto con los comandos
Salida	int: 0 si no hay información en memoria, -1 si hay un error al escribir, 1 si es exitoso
Precondiciones	Debe haber información cargada en memoria según el tipo solicitado

Postcondiciones	El archivo queda creado/sobrescrito con el contenido en memoria en formato compatible con cargarComando/cargarElementos
------------------------	---

D. Agregar Movimiento

Campo	Descripción
Firma	Int agregarMovimiento (std::queue<Comando>& comandos, const std::string tipoMovimiento, double magnitud, const std::string& unidad)
Entradas	Comandos: Referencia a lista de comandos actual tipoMovimiento: Cadena de caracteres que define el tipo de movimiento (Avanzar, Girar) Magnitud: El valor por el cual se va a modificar el movimiento Unidad: Unidad en la cual se aplica la operación de movimiento
Salida	int: -1 si falla la conversión de unidades, 1 si funciona
Precondiciones	Información introducida por el usuario
Postcondiciones	Se modifica la lista de comandos, agregando un nuevo comando de movimiento .

E. Agregar Analisis

Campo	Descripción
Firma	int agregarAnalisis(std::queue<Comando>& comandos, const std::string& tipoAnalisis, const std::string& objeto, const std::string& comentario = "")
Entradas	Comandos: Referencia a lista de comandos actual tipoAnalisis: Cadena de caracteres que define el tipo de analisis(fotografiar, composición, perforar) Objeto: Cadena de caracteres que define el objeto Comentario: Cadena de caracteres que define el contenido del comentario
Salida	int: -1 si el tipo de análisis no está en los estipulados, 1 si funciona
Precondiciones	Información introducida por el usuario
Postcondiciones	Se modifica la lista de comandos, agregando un nuevo comando de analisis.

F. Agregar Elemento

Campo	Descripción
Firma	int agregarElemento(std::vector<Elemento>& elementos, std::string tipoElem, double tamano, std::string unidad, double x, double y)
Entradas	Elementos: Arreglo de elementos TipoElem: El tipo de elemento a agregar Tamano: Tamaño del elemento a agregar Unidad: Unidad del tamaño X: coordenada x Y: coordenada y
Salida	int: -1 si el tipo de elemento no está en los estipulados, 1 si funciona
Precondiciones	Información introducida por el usuario
Postcondiciones	Se modifica el arreglo de elementos, agregando el elemento nuevo.

G. Simular Comandos

Campo	Descripción
Firma	int simularComandos(std::queue<Comando> comandos, double x, double y, double &nuevo_x, double &nuevo_y)
Entradas	Comandos: Lista de comandos actuales X: posición actual en x Y: posición actual en y Nuevo_x: valor resultante en x de la simulación Nuevo_y: valor resultante en y de la simulación
Salida	int: 1 si funciona
Precondiciones	Lista de comandos llena con los comandos a simular
Postcondiciones	Modifica nuevo_x y nuevo_y con los valores resultantes de la simulación

H. Parser

Campo	Descripción
Firma	vector<string> parser(const string& linea)
Entradas	Linea: Cadena de caracteres que va a ser parseada
Salida	Vector de strings que contiene los elementos de la línea separados por un espacio.
Precondiciones	Input del usuario
Postcondiciones	Vector de string que representa los comandos del usuario

I. Extraer Comentario

Campo	Descripción
Firma	string extraerComentario(const string& linea)
Entradas	Linea: Cadena de caracteres a la cual se va a extraer el comentario
Salida	String que contiene el comentario
Precondiciones	Input del usuario
Postcondiciones	String con el comentario

J. Intersecta

Campo	Descripción
Firma	bool Rectangulo::intersecta(double x1, double x2, double y1, double y2) {
Entradas	x1: coordenada mínima en x a evaluar x2: coordenada máxima en x a evaluar y1: coordenada mínima en y a evaluar y1: coordenada máxima en y a evaluar
Salida	Bool: Retorna true si el área objetivo se solapa con el área definida por el rectángulo. Retorna false si por el contrario no está contenida.
Precondiciones	Input del usuario debe estar en el formato adecuado.
Postcondiciones	N/A

K. En cuadrante

Campo	Descripción
Firma	bool Rectangulo::en_cuadrante(double x1, double x2, double y1, double y2) {
Entradas	x1: coordenada mínima en x a evaluar x2: coordenada máxima en x a evaluar y1: coordenada mínima en y a evaluar y1: coordenada máxima en y a evaluar
Salida	Una lista de elementos, que contiene los elementos que están contenidos dentro del área definida por los parámetros.
Precondiciones	Las coordenadas deben definir un área válida.
Postcondiciones	N/A

L. Djisktra

Campo	Descripción
Firma	void Grafo::dijkstra(Elemento origen, std::map<Elemento, double>& dist, std::map<Elemento,Elemento>& previo)
Entradas	Origen: Elemento desde donde se decide comenzar Dist: Mapa que relaciona elemento con su distancia respecto al origen Previo: Mapa que guarda la secuencialidad en la que los elementos son recorridos
Salida	Funcion void, modifica el arreglo de di
Precondiciones	Las coordenadas deben definir un área valida.
Postcondiciones	N/A

M. Nodo mas lejano

Campo	Descripción
Firma	Elemento Grafo::nodo_mas_lejano(std::map<Elemento,double>& dist)
Entradas	Dist: mapa de distancias de los elementos
Salida	Elemento mas lejano
Precondiciones	Lista de dist llena
Postcondiciones	N/A

N. Reconstruir ruta

Campo	Descripción
Firma	std::vector<Elemento> Grafo::reconstruir_ruta(Elemento origen,Elemento destino,std::map<Elemento,Elemento>& previo)
Entradas	Origen: elemento de origen Destino: elemento de destino Previo: mapa del orden de el camino mas largo
Salida	Vector secuencial en orden
Precondiciones	Dijkstra
Postcondiciones	N/A

O. Ruta más larga

Campo	Descripción
Firma	void Grafo::ruta_mas_larga()
Entradas	N/A
Salida	N/A
Precondiciones	Construcción del grafo
Postcondiciones	N/A

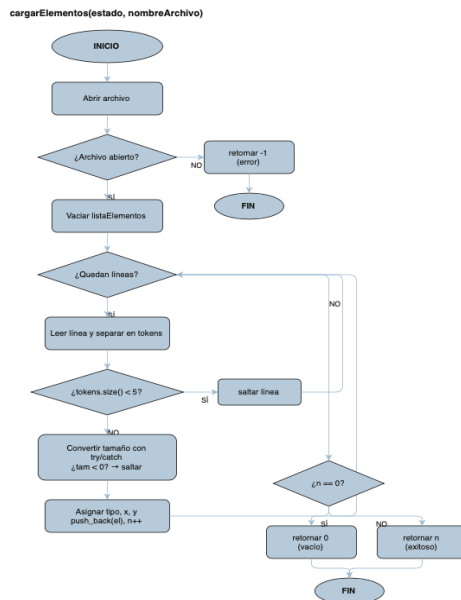
P. N_vecinos

Campo	Descripción
Firma	std::vector<Elemento> QuadTree::n_vecinos(Elemento elemento,int n)
Entradas	Elemento: elemento N: numero de vecinos a obtener
Salida	Vector de n elementos mas cercanos
Precondiciones	Quadtree
Postcondiciones	N/A

4. Diagramas de flujo

Figura 2.

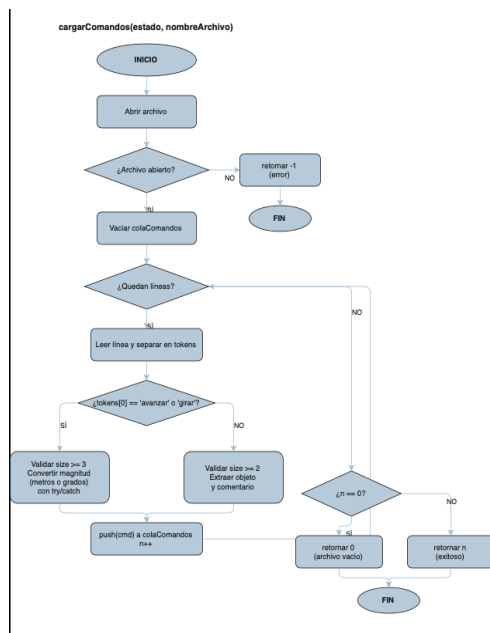
Diagrama de Flujo — cargarElementos



Fuente: Elaboración propia mediante *app.diagrams.net*

Figura 3.

Diagrama de Flujo — cargarComandos



Fuente: Elaboración propia mediante *app.diagrams.net*

Figura 4.

Diagrama de Flujo — guardar

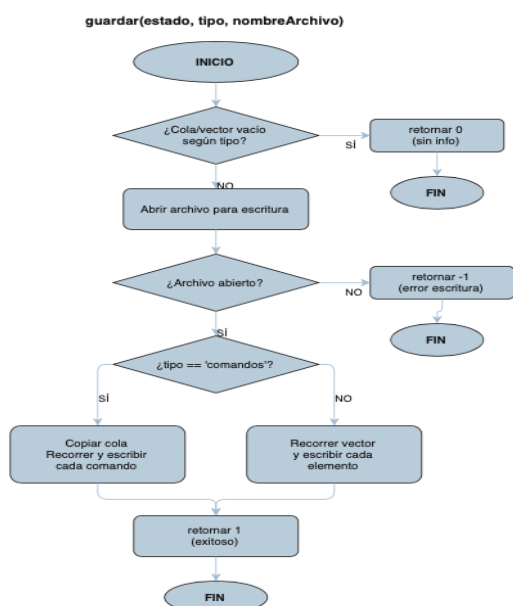
Fuente: Elaboración propia mediante *app.diagrams.net***Figura 5.**

Diagrama de Flujo — simularComandos

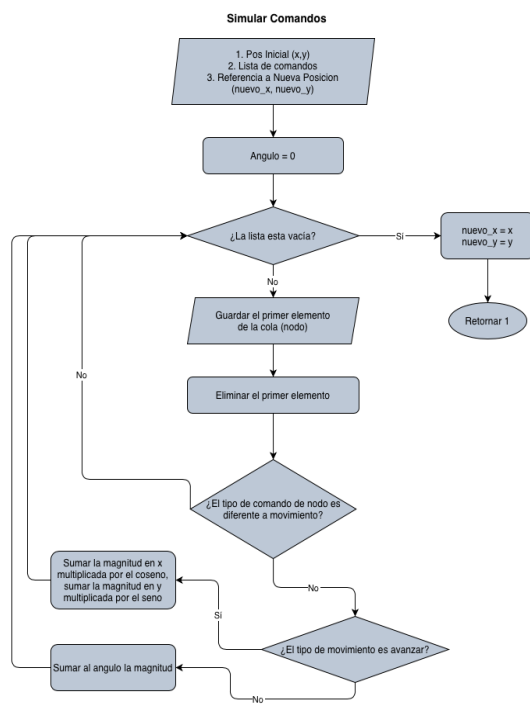
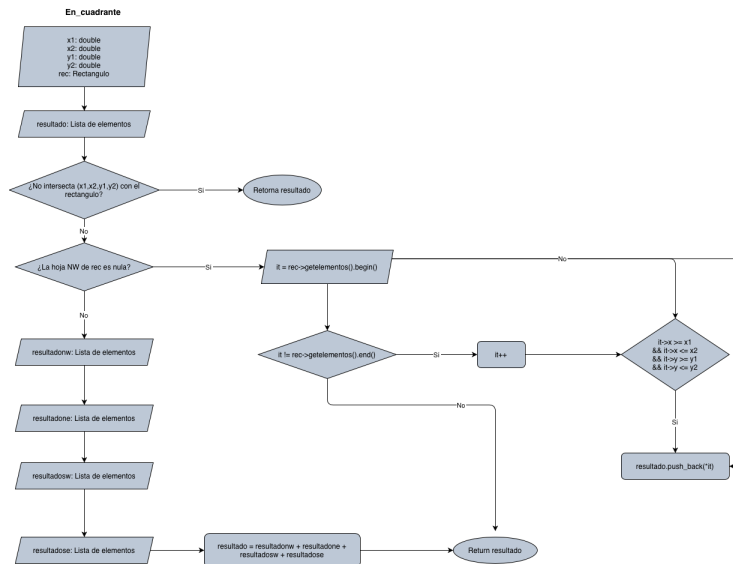
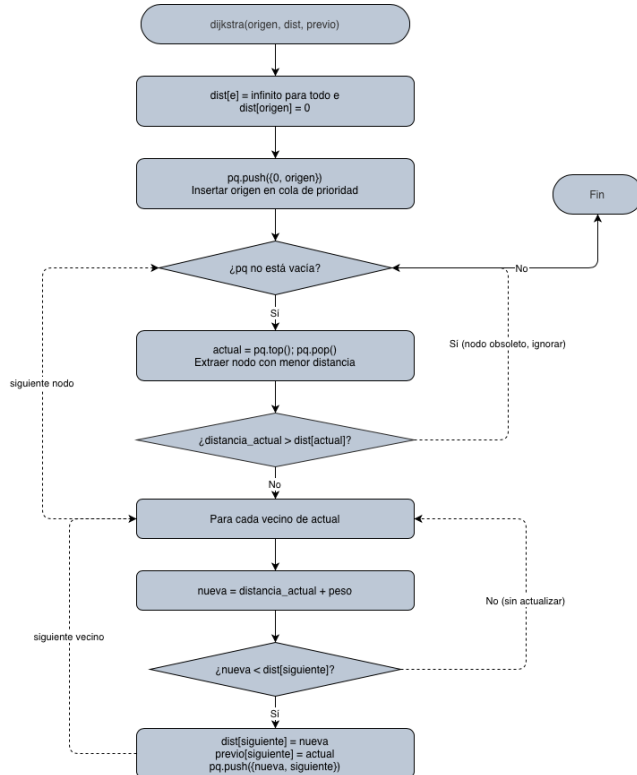
Fuente: Elaboración propia mediante *app.diagrams.net*

Figura 6.**Diagrama de Flujo — en_Cuadrante**

Fuente: Elaboración propia mediante *app.diagrams.net*

Figura 7.**Diagrama de Flujo — dijkstra**

Fuente: Elaboración propia mediante *app.diagrams.net*

Figura 8.

Diagrama de Flujo — nodo_mas_lejano

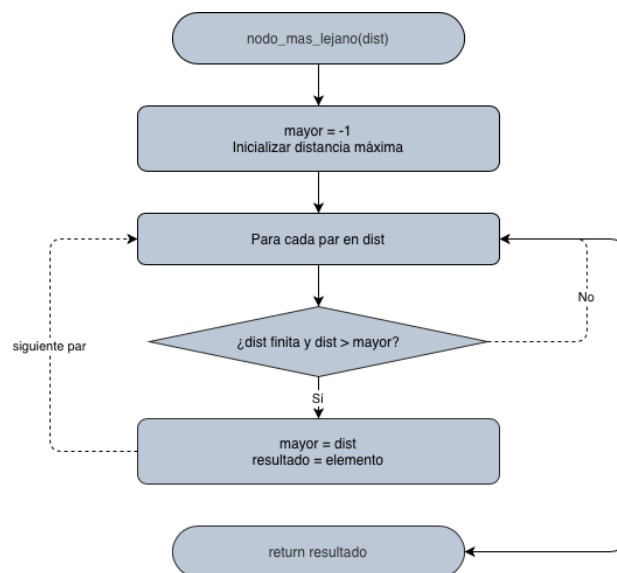
Fuente: Elaboración propia mediante *app.diagrams.net***Figura 9.**

Diagrama de Flujo — reconstruir_ruta

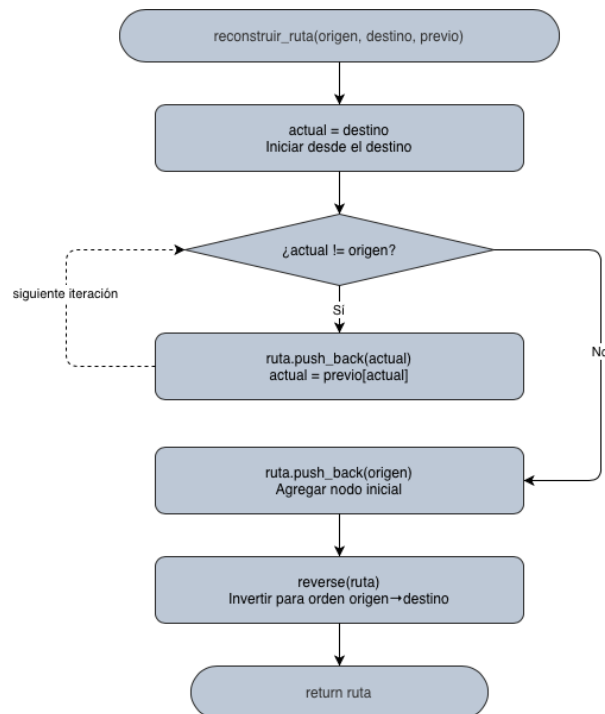
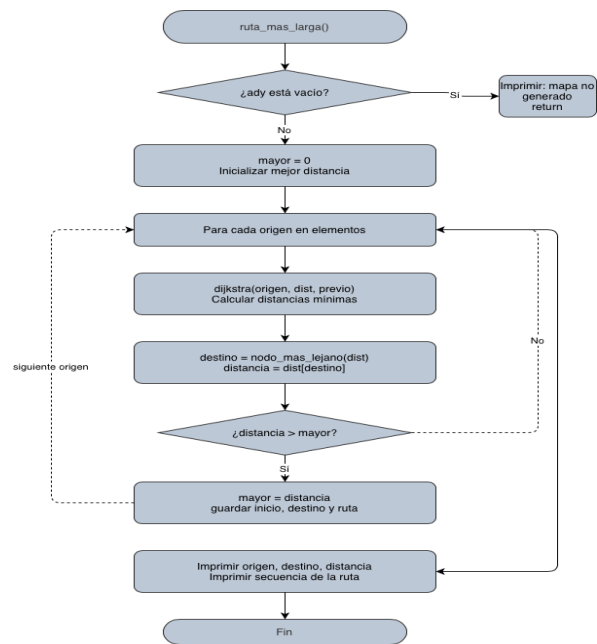
Fuente: Elaboración propia mediante *app.diagrams.net*

Figura 10.**Diagrama de Flujo — ruta_mas_larga**

Fuente: Elaboración propia mediante *app.diagrams.net*

5. Plan de pruebas

1. Plan de Pruebas simular_comandos

Plan de pruebas: funcion simular_comandos			
Descripción del caso	Valores de entrada	Resultado esperado	Resultado obtenido
1. Archivo de comandos con comandos de movimiento variados (casoprueba1.txt)	posX =0 y posY = 0	(0.2334, 0.2811)	(0.233371, 0.281126)
2. No se cargan comandos	posX=1 y posY=10	(1,10)	(1,10)
3. Archivo de comandos con solo comandos de tipo análisis (casoprueba3.txt)	posX=6 y posY=10	(6,10)	(6,10)
4. Archivo de comandos con comandos de analisis y movimientos mezclados (casoprueba4.txt)	posX=100 posY=200	(100.2817, 200.2682)	(100.282, 200.268)

La demostración de estos resultados se encuentra en el archivo demostracion.pdf adjuntado en la carpeta.

2. Plan de pruebas en _cuadrante

Todos estos casos de pruebas se realizarán cargando los elementos del casoprueba2.txt

Plan de pruebas: funcion en _cuadrante			
Descripción del caso	Valores de entrada	Resultado esperado	Resultado obtenido
1. Caso vacio	$x1 = 0, x2 = 0,$ $y1=0, y2=0$	Error	Error
2. Caso general (caso prueba2.txt)	$x1 = 1, x2 = 5,$ $y1=1, y2=5$	roca 3 (2, 2) monticulo 4 (5,5) crater 4 (4,1) roca 5 (1,3)	Montículo tamaño 4 (5,5) Roca tamaño 5 (1,3) Roca tamaño 3 (2,2) Crater tamaño 4 (4,1)
3. Caso con ningún elemento dentro de la región(casoprueba5.txt)	$x1 = 2, x2 = 6,$ $y1=2, y2=6$	No retorna ningún elemento	Los elementos ubicados en cuadrante solicitado son: 0
4. Elementos en los bordes (casoprueba6.txt)	$x1 = 2, x2 = 6,$ $y1=2, y2=6$	Cráter tamaño 1 (4,4) Montículo tamaño 1 (2,6) Duna tamaño 1 (6,6) Roca tamaño 1 (2,2)	(Resultado exitoso) Los elementos ubicados en cuadrante solicitado son: 4 Crater tamaño 1 (4,4) Monticulo tamaño 1 (2,6) Duna tamaño 1 (6,6) Roca tamaño 1 (2,2)

3. Plan de pruebas ruta_*mas*_larga

Todos estos casos de pruebas se realizarán cargando los elementos del casoprueba6.txt

Plan de pruebas: funcion en _cuadrante			
Descripción del caso	Valores de entrada	Resultado esperado	Resultado obtenido
5. Mapa con coeficiente 0.5	N/A	(2,2) roca (4,4) crater (6,6) Duna	Ruta: - (2.000000,2.000000,Roca) - (4.000000,4.000000,Crater) - (6.000000,6.000000,Duna)
Mapa con coeficiente 0.3	N/A	(2,2) roca (4,4) crater (2, 6) monticulo	-(2.000000,2.000000,Roca) - (4.000000,4.000000,Crater) - (2.000000,6.000000,Monticulo)